Object Oriented Analysis and Design

State Machine Diagrams

Understanding State

State: A combination of values for an object's properties at a specific point in time. State determines how an object responds to events.

Example: Traffic Light Object

- Properties→ currentColor: {Red, Yellow, Green}
- Properties→ timer: integer (seconds remaining)
- Possible States → (Red, 30): Red light with 30 seconds left
- Possible States → (Yellow, 3): Yellow light with 3 seconds left
- Possible States → (Green, 45): Green light with 45 seconds left

Challenge Question 1

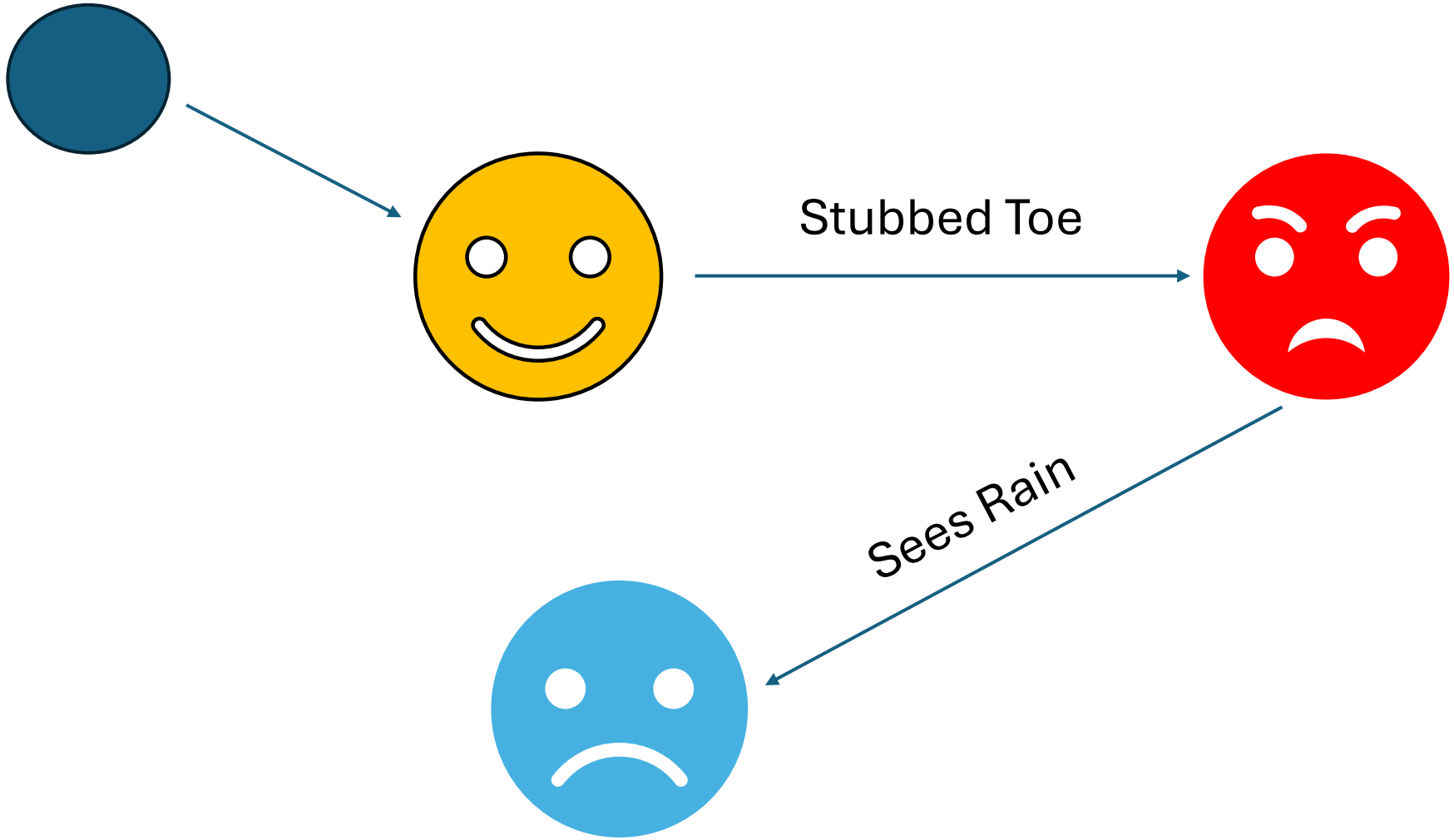
Which of the following best describes why state is important in OOP?

- A) It determines the memory allocation for the object
- B) It defines which methods are available in the class
- C) It controls how an object responds to operations and events based on current conditions
- D) It specifies the inheritance hierarchy of the class.

Consider a human having 3 emotional states

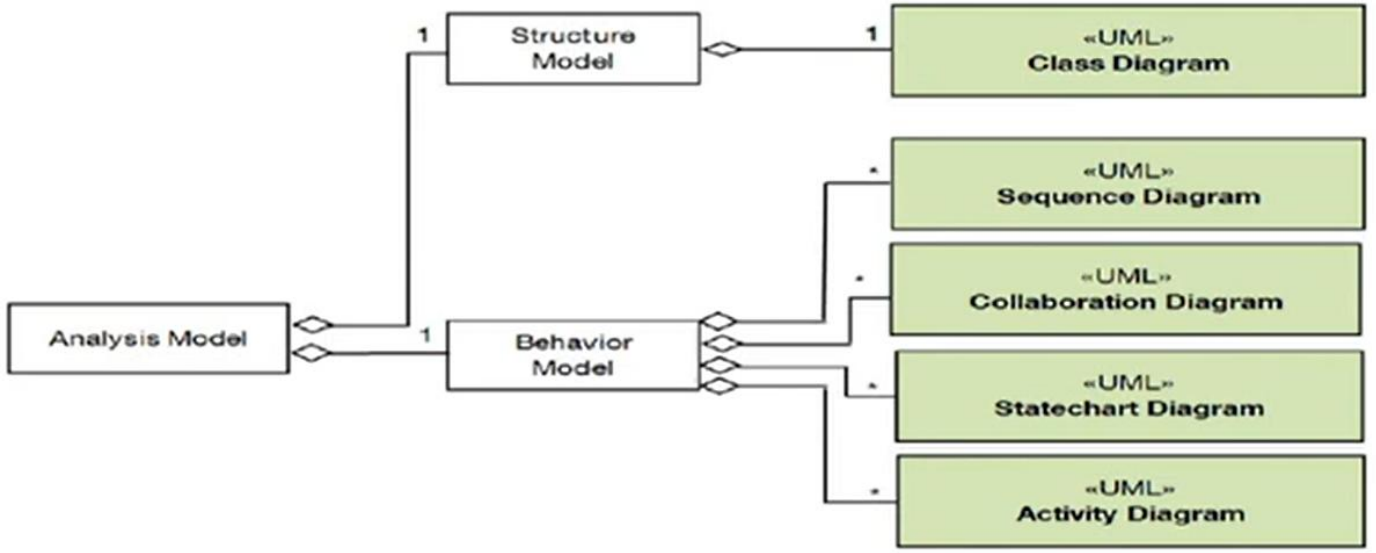


State diagram

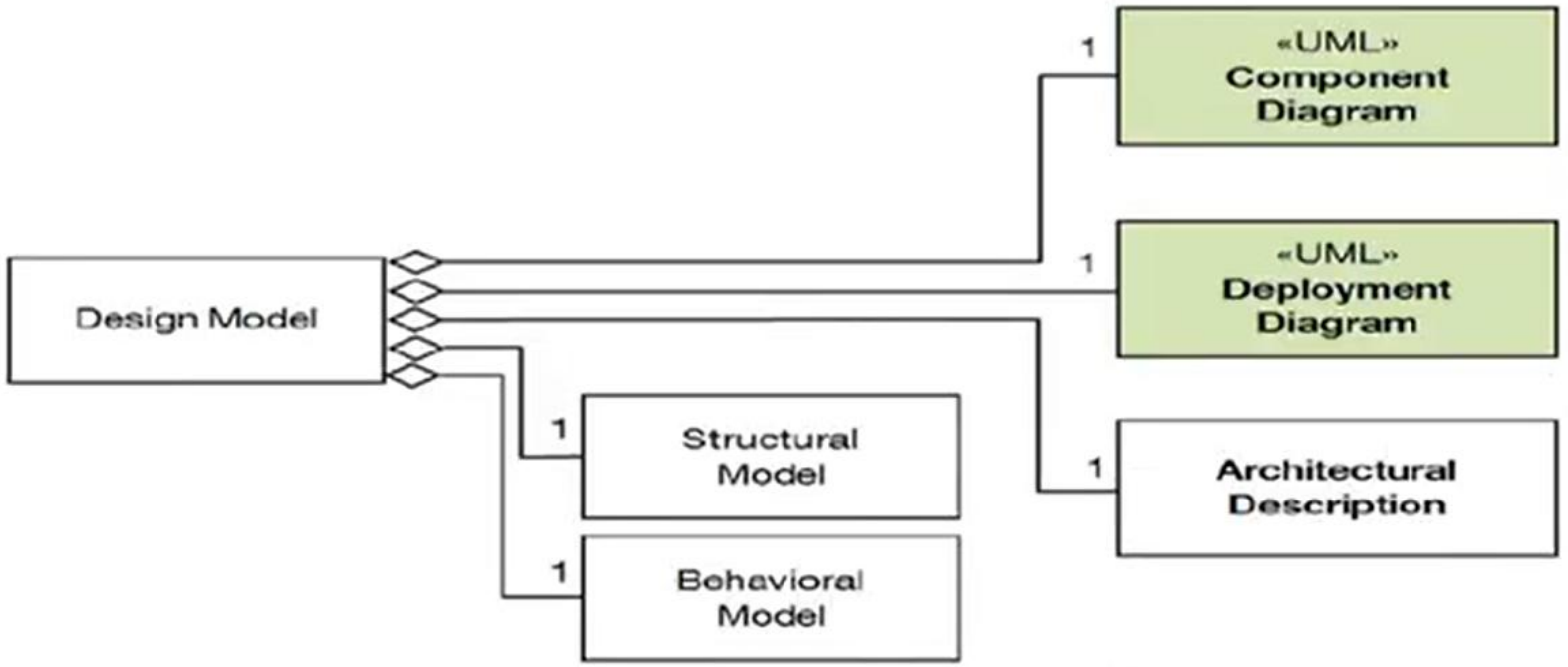


State Machine Diagrams in SDLC Phase

- In the analysis phase, the problem domain is analyzed and refined from the Requirements phase
- The behavior model of the system is hence understood in this phase
- State machine diagrams is a major result of the Analysis Phase



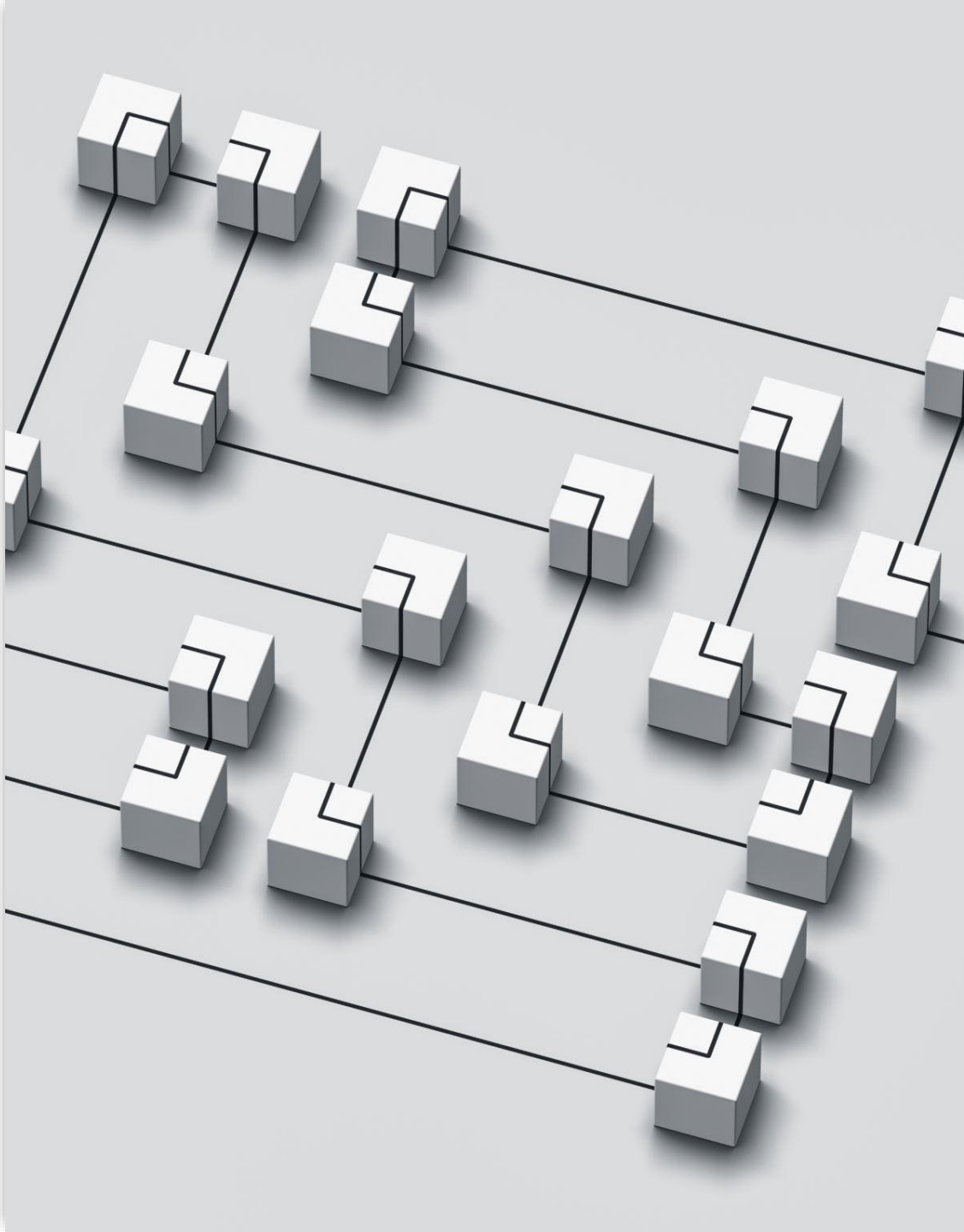
State Machine Diagrams in SDLC Phases



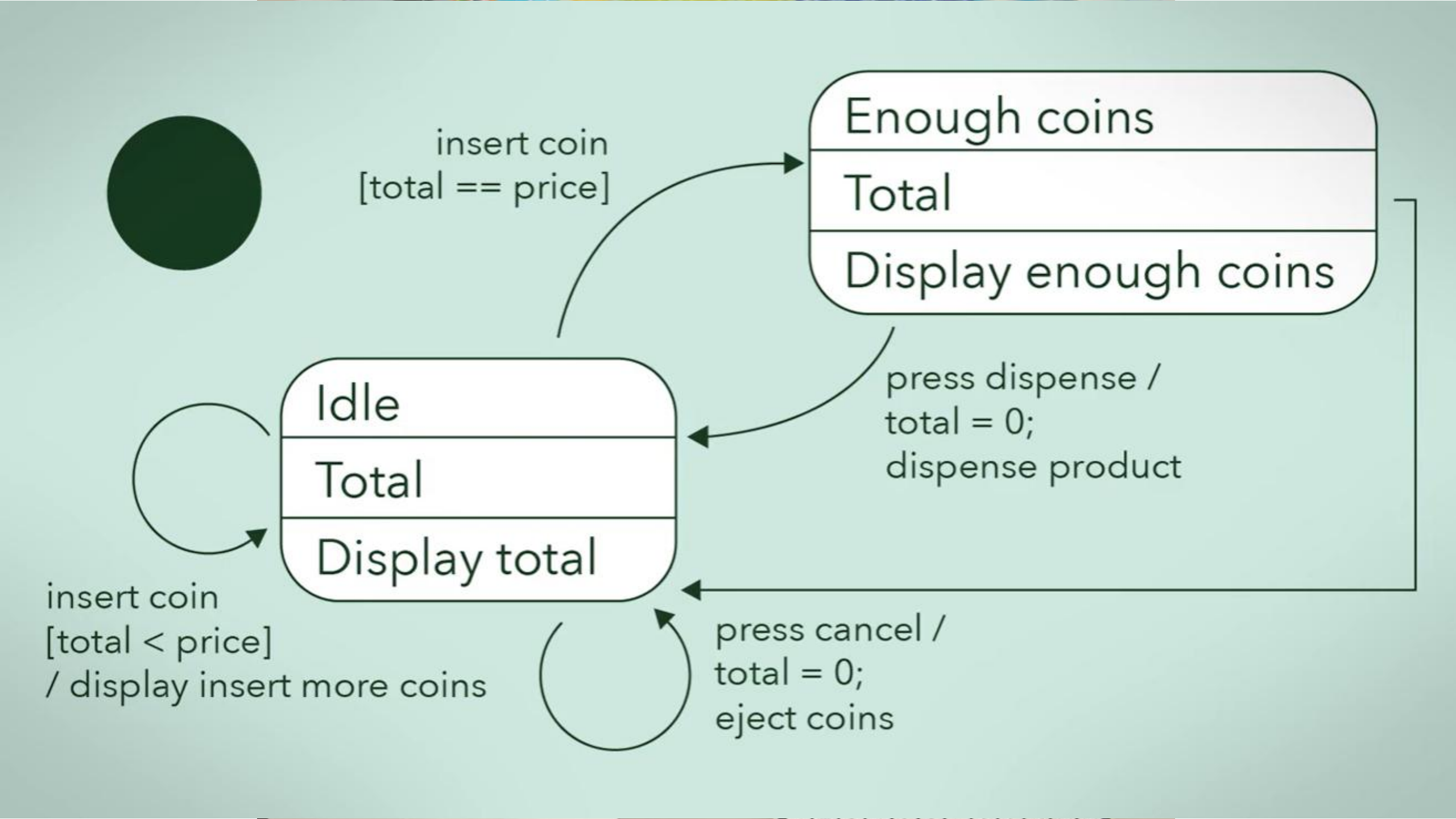
- It is further refined in the Design phase.

What are State Machine Diagrams

- State Machine Diagram is a behavior diagram that shows discrete behavior through finite state transitions. It models how an object moves through different states in response to events.
  - Behavioral State Machine
  - Protocol State Machine




KIA


Enough coins
Total
Display enough coins
insert coin
[total == price]
press dispense /
total = 0;
dispense product
Idle
Total
Display total
insert coin
[total < price]
/ display insert more coins
press cancel /
total = 0;
eject coins

Types of State Machines

Behavioral State Machine	Protocol State Machine
What does it do?	What is allowed to be called?
Can have entry/exit/do activities	Cannot have entry/exit/do activities
Used for implementation like Behaviour, actions, effects	Used for specifying valid operation sequences
Example: How a bank account processes transactions	Example: which operations are legal in each account state

Behavioral State Machine Components

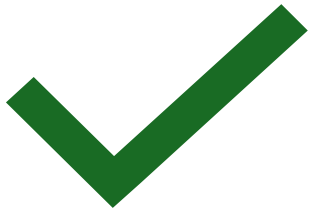


Core Components

Vertex: Nodes in the state machine graph (states and pseudostates)

Transitions: Directed edges connecting vertices

Context: The class that owns this state machine



Behavioral State Properties

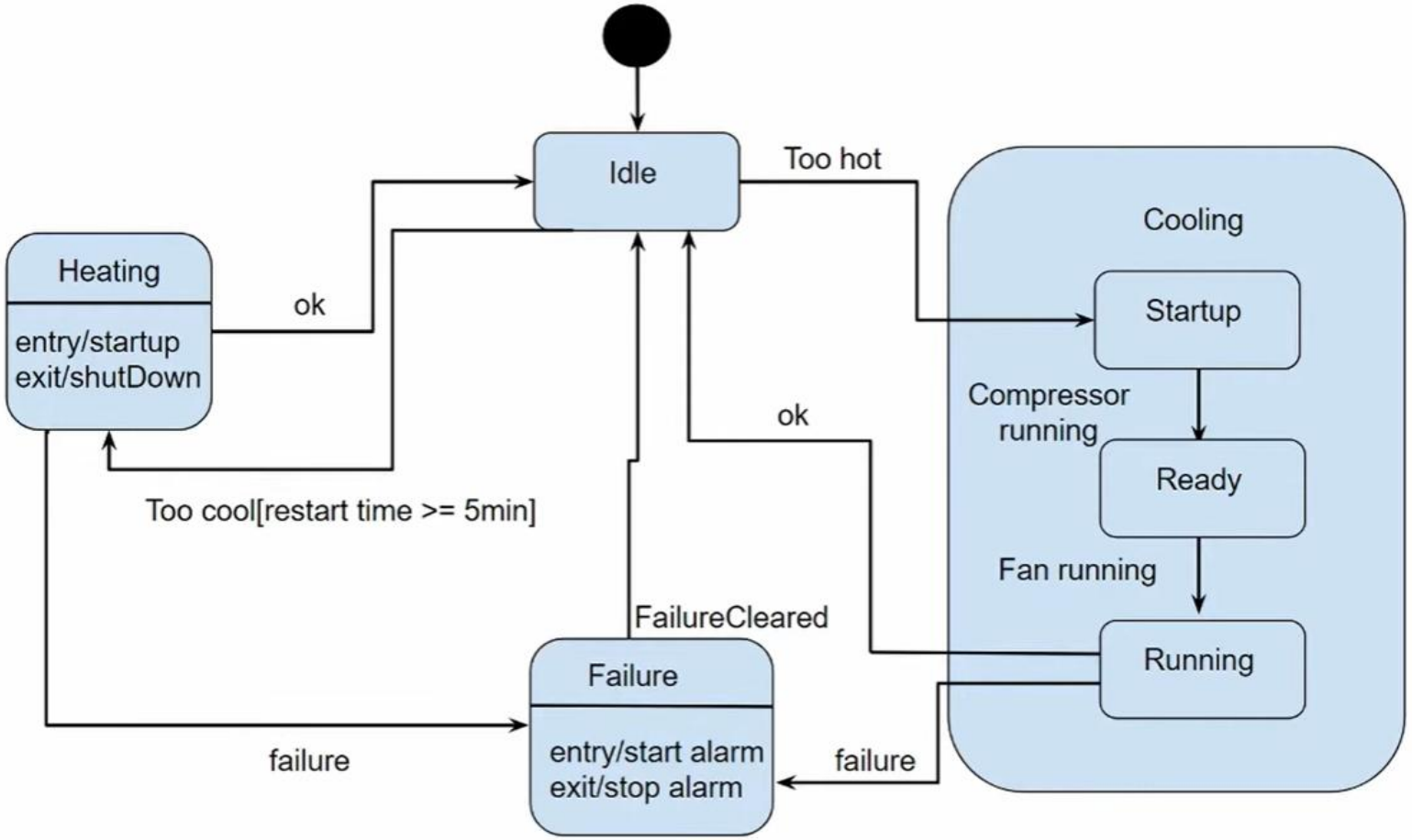
Name: Identifier for the state

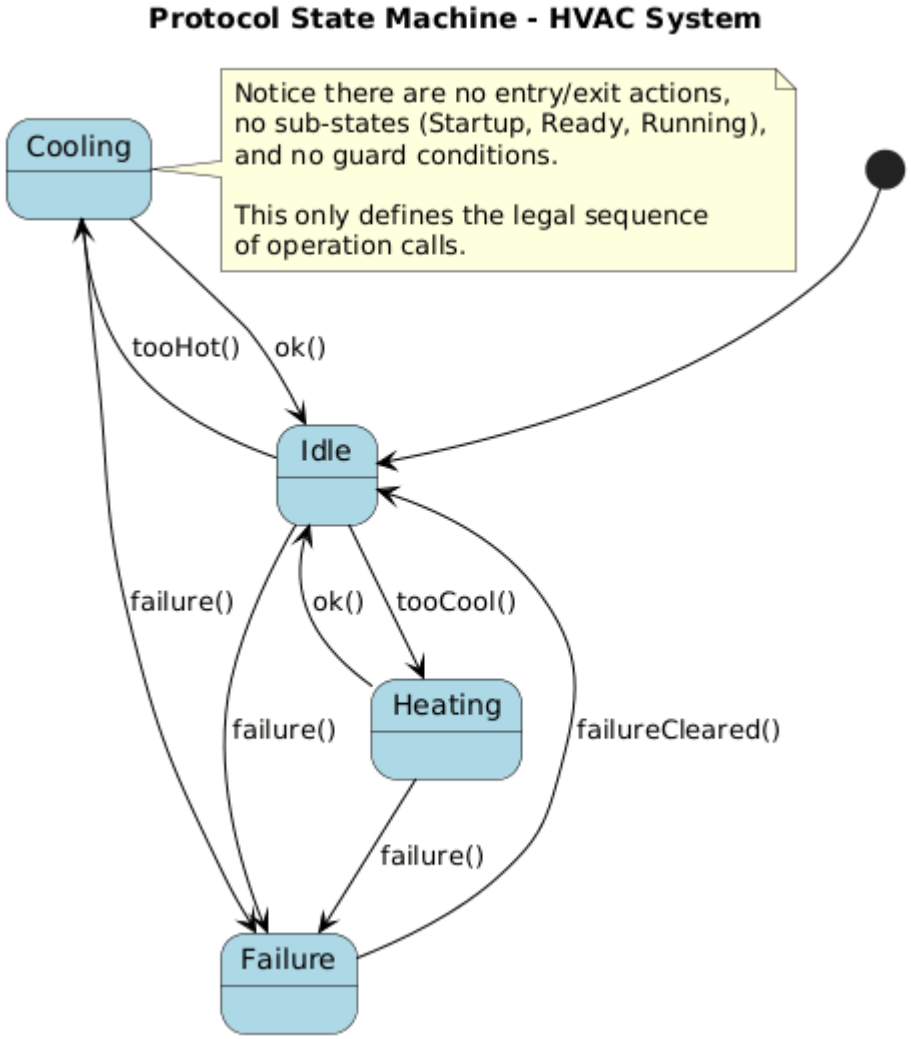
Entry Action: Executed when entering the state

Exit Action: Executed when leaving the state

Do Activity: Ongoing Behavior while in the state

Internal Transitions: Events handled without leaving the state





Behavioral State Machine

Simple State

A state with no substates. It’s atomic and cannot be decomposed further.

Simple state is shown as a rectangle with rounded corners and the state name inside the rectangle.

Simple state may have compartments. The compartments of the state are:

Name compartment	Internal activities compartment	Internal transitions compartment

Simple state



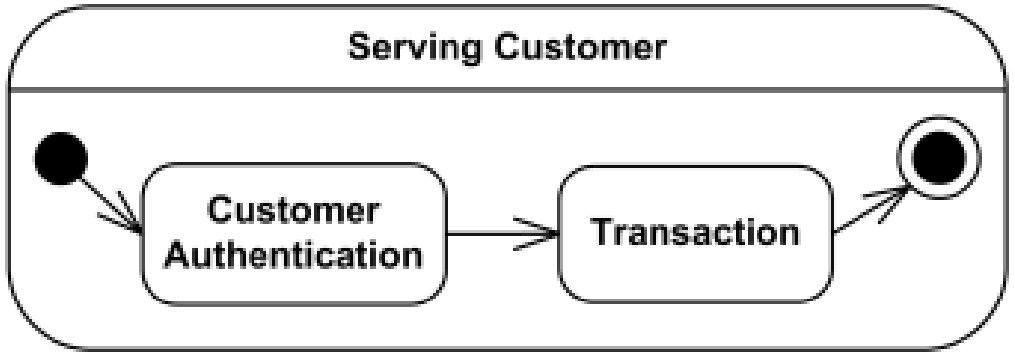
Simple state Waiting for Customer Input.



Simple state ***Waiting for Customer Input*** *with name and internal activities compartments.*

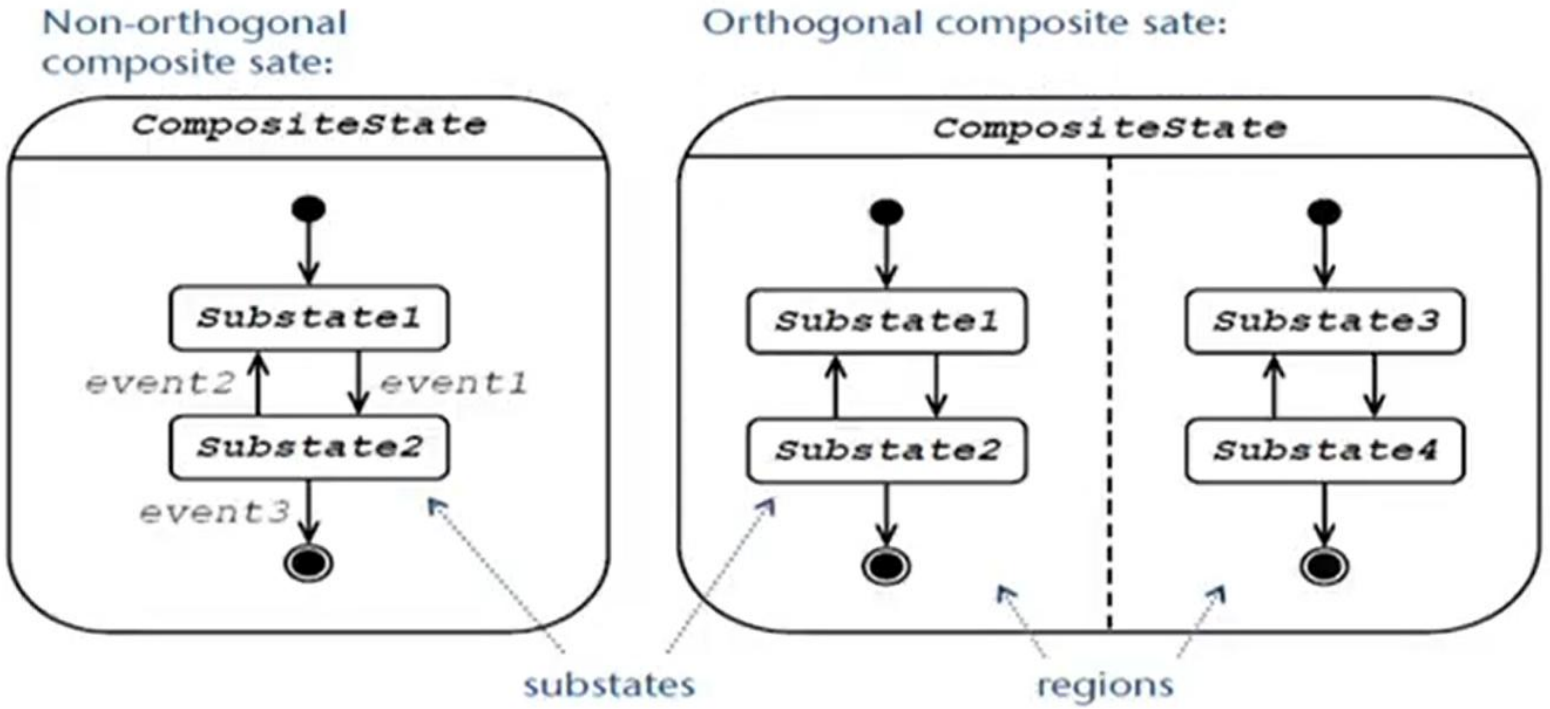
Composite State

- A state that contains other states (substates)
  - Sequential/Disjoint: Only one substate is active at a time
  - Concurrent/Orthogonal: Multiple substates active simultaneously (separated by dashed lines called regions)

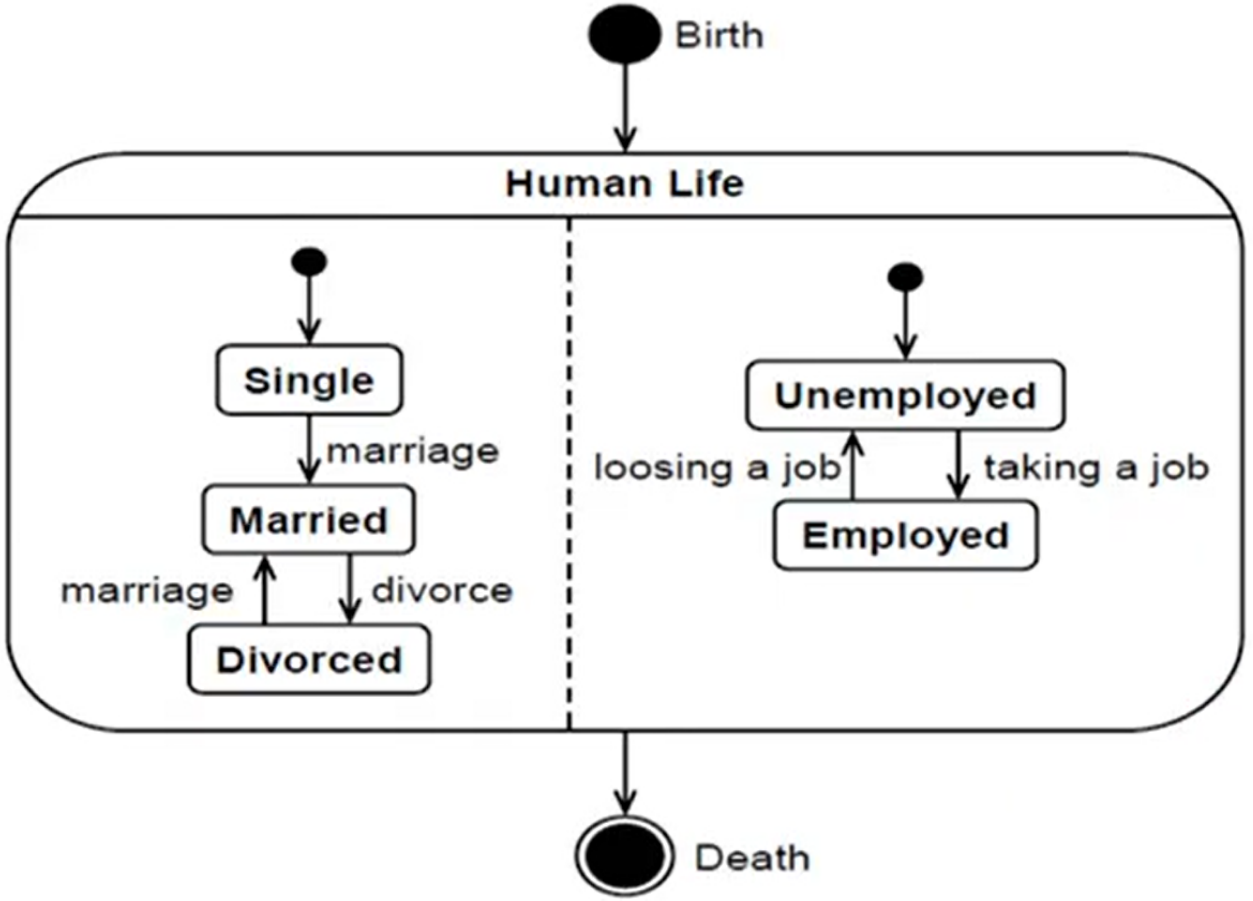


Simple composite state Serving Customer has two substates.

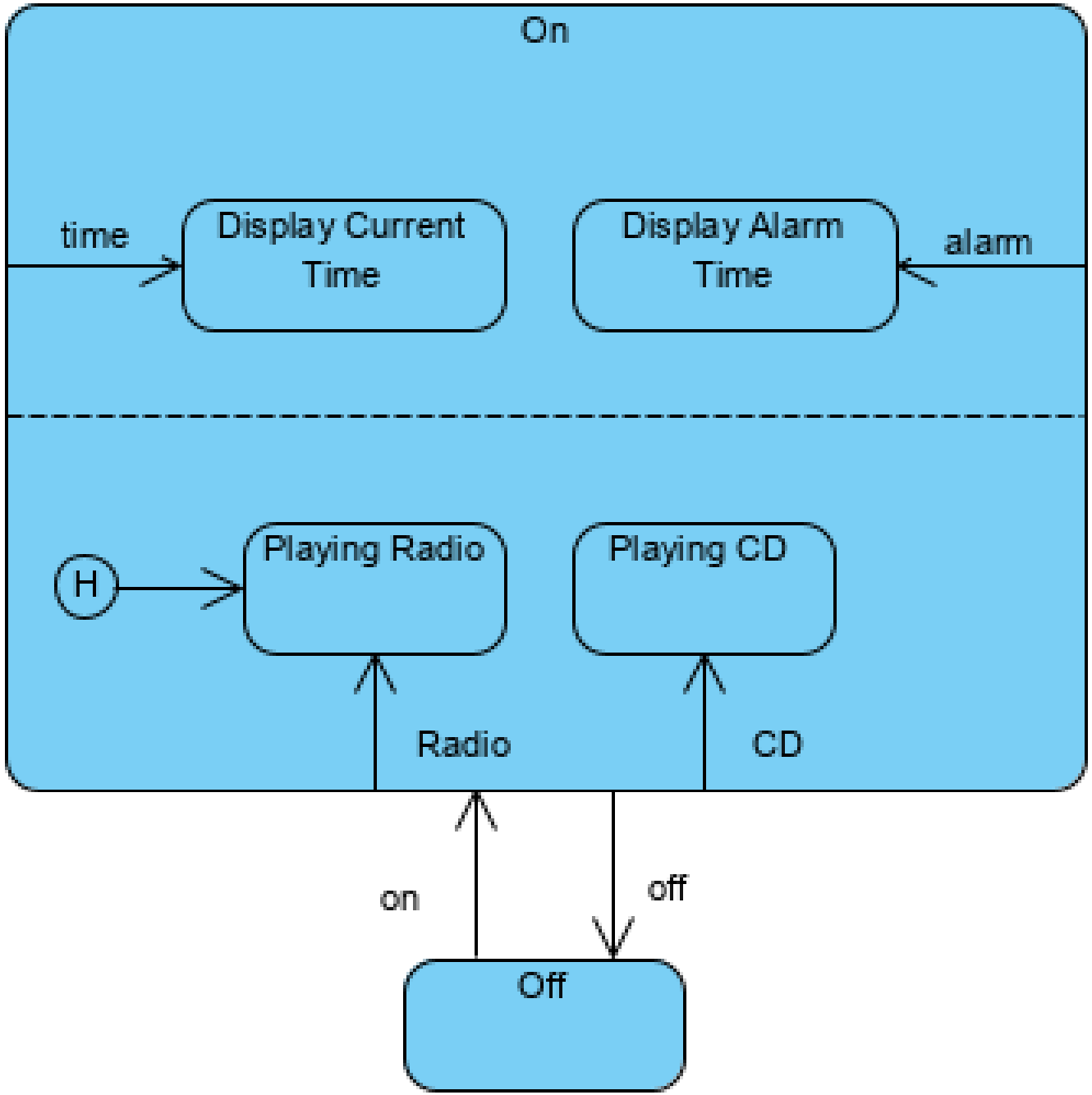
Composite State



Composite State



Concurrent Sub-States and Regions



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/about-state-diagrams/

Submachine State

Think of a submachine state as a “function” or a “plug-in” in programming. Instead of drawing a huge, messy diagram, you draw the complex details on a separate page and just put a box that says, “Refer to the other diagram”.

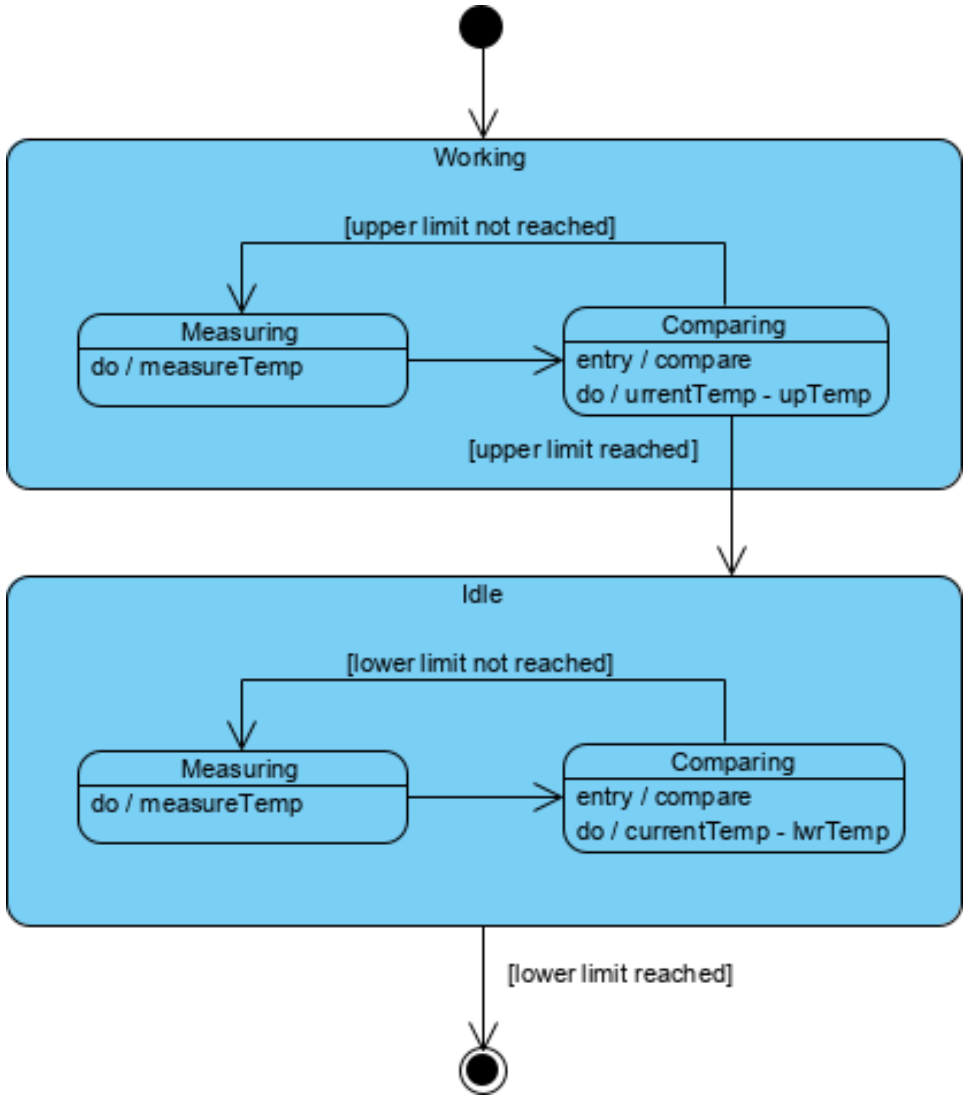
A submachine state acts like a “plug-in” box. It tells the system to run a separate, detailed state machine inside the current state.

You can reuse the exact same “plug-in” box multiple times inside one main diagram. You don’t have to redraw it every time you need it.

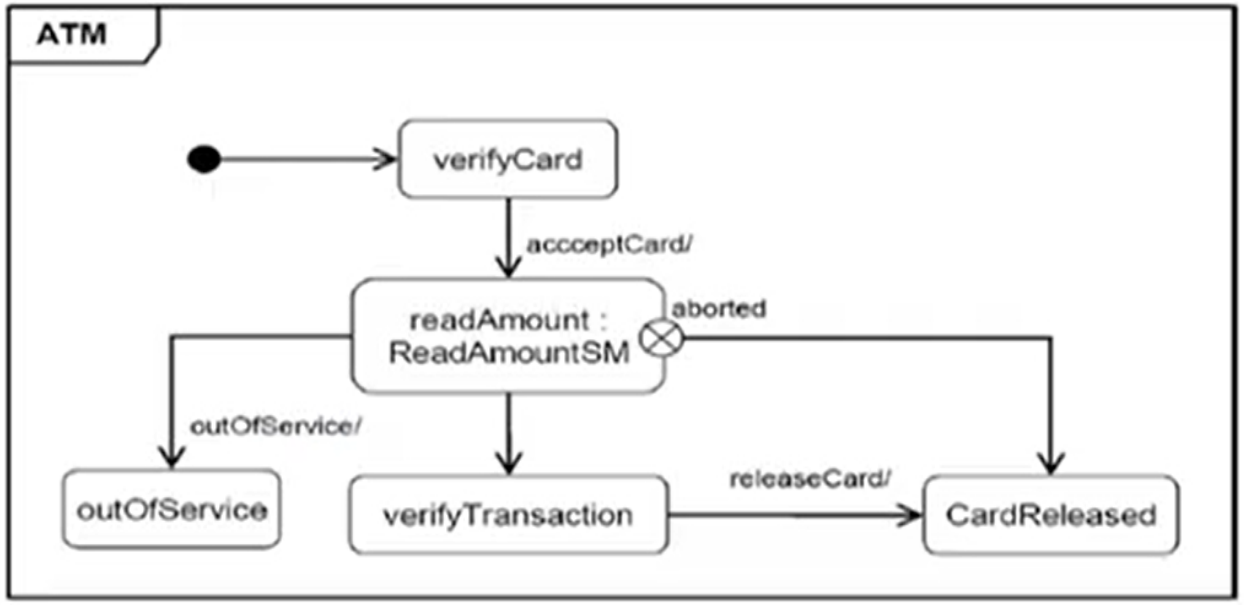
It helps break down complex systems into small, reusable pieces. You write the logic once, and reuse it wherever you need it, keeping your main diagram clean and easy to read.

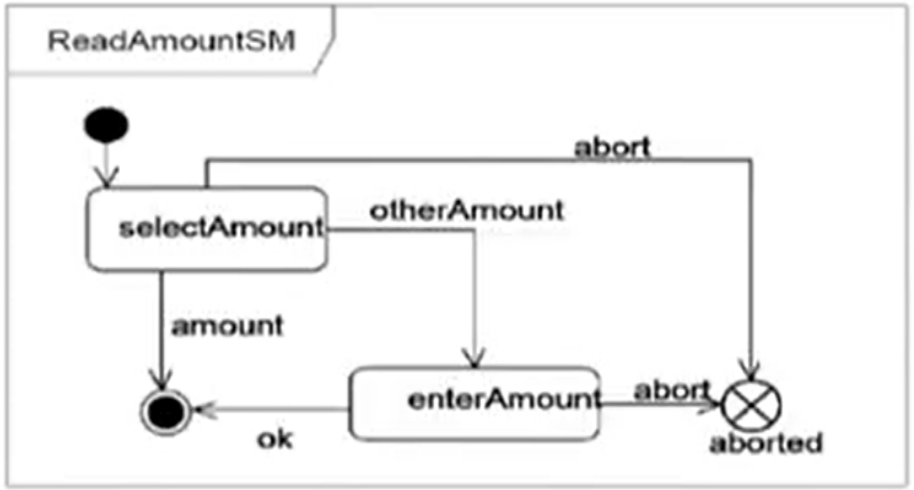
Submachine State

- References another state machine diagram, allowing reuse of common behavior patterns across different contexts.

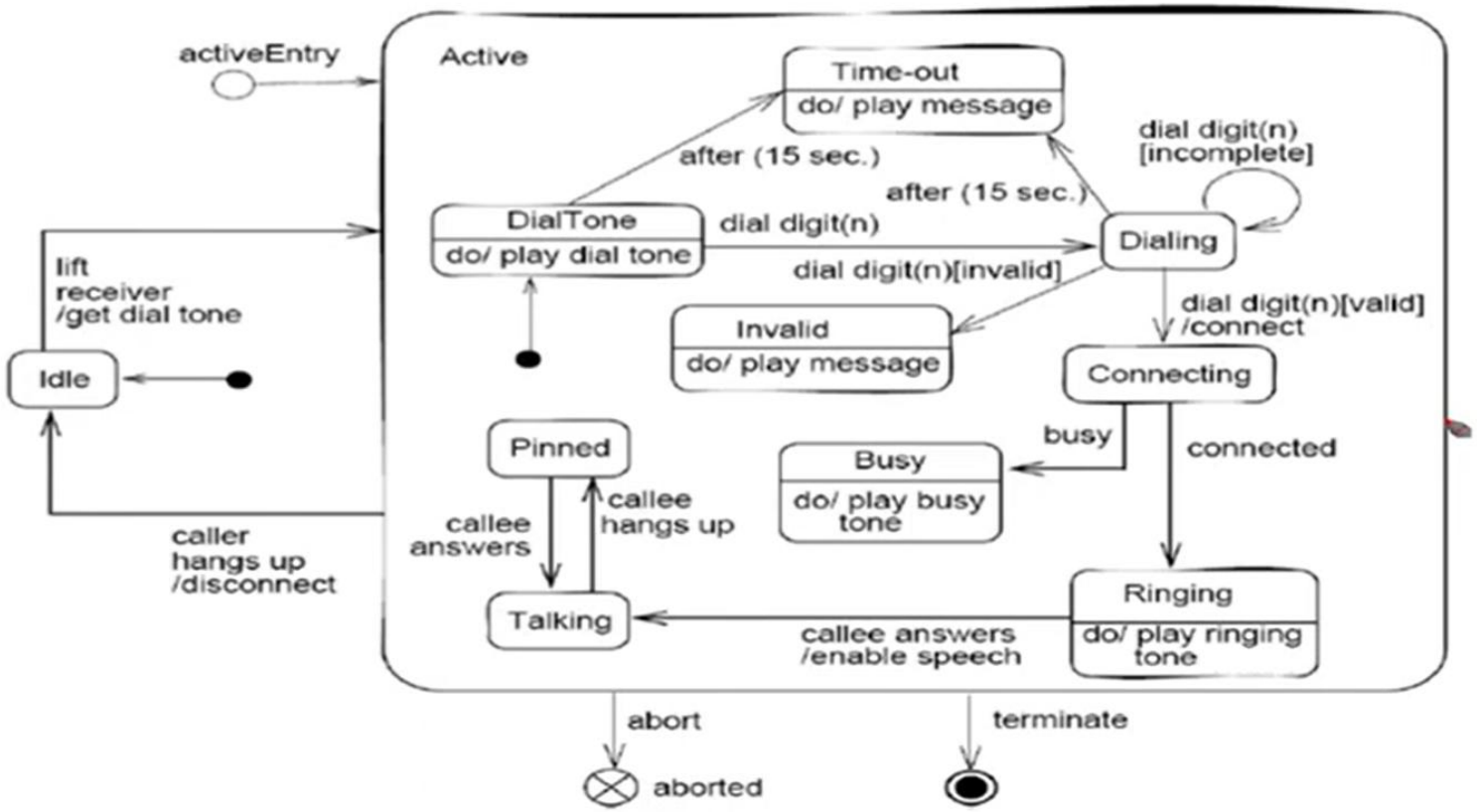


Submachine State





Behavioral State Machine



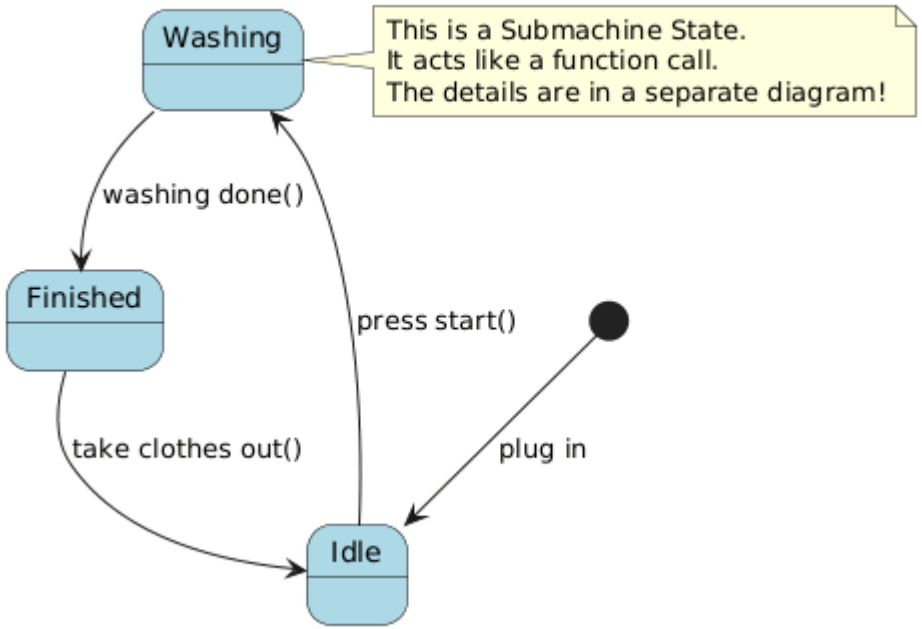
Behavioral state machine for Dialing a Phone



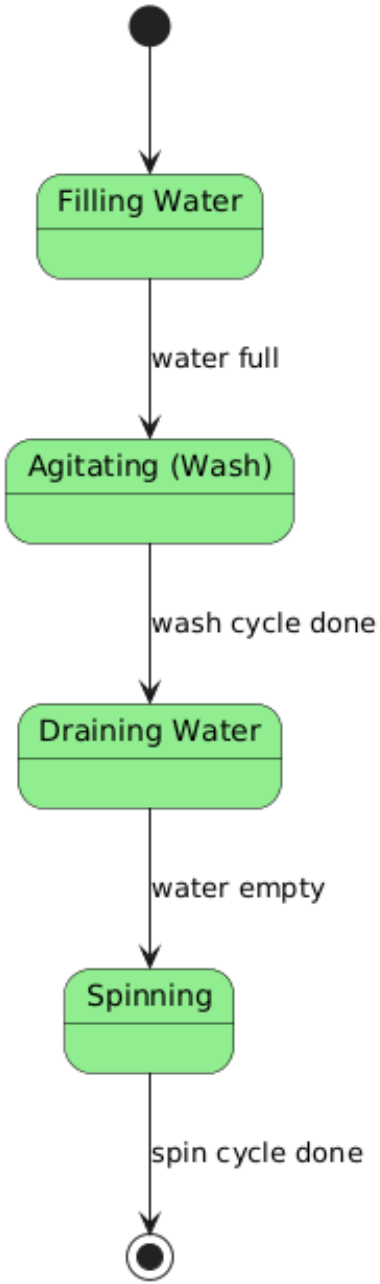
Quick Think

- Can you suggest a scenario where you'd need a composite state with concurrent regions?

Main State Machine: Washing Machine



Submachine: The Washing Process



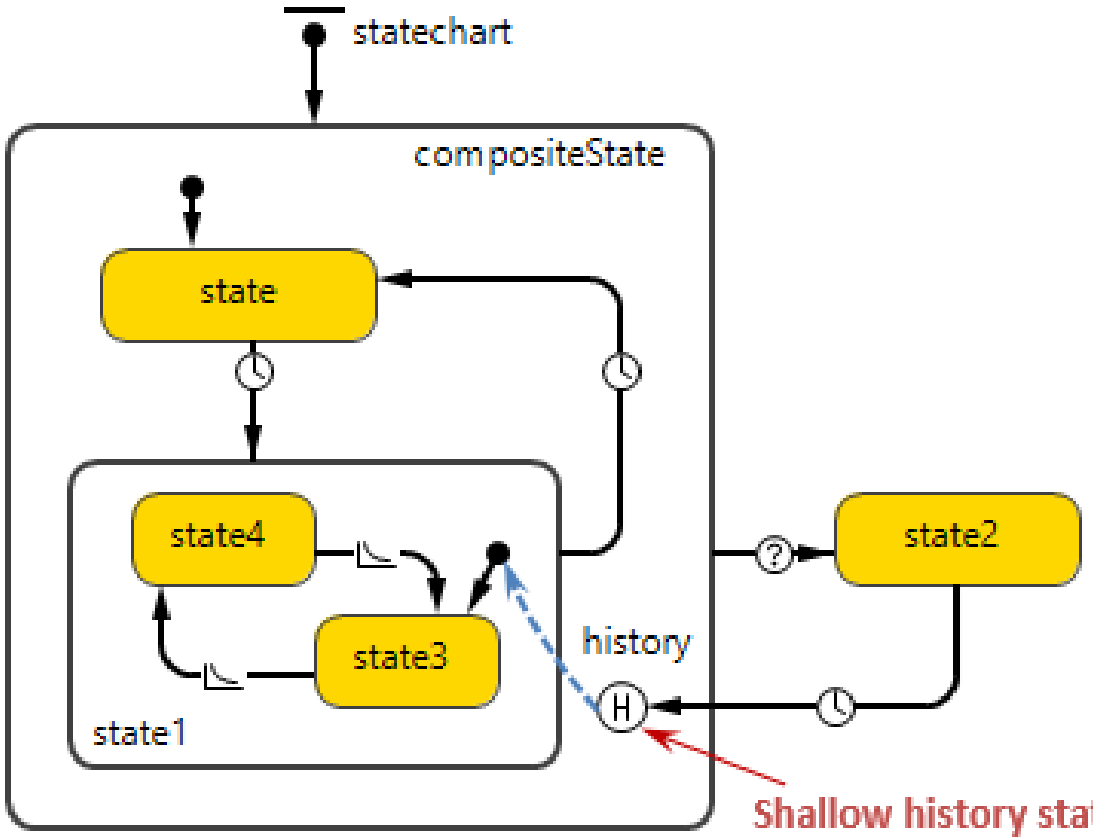
Pseudostate

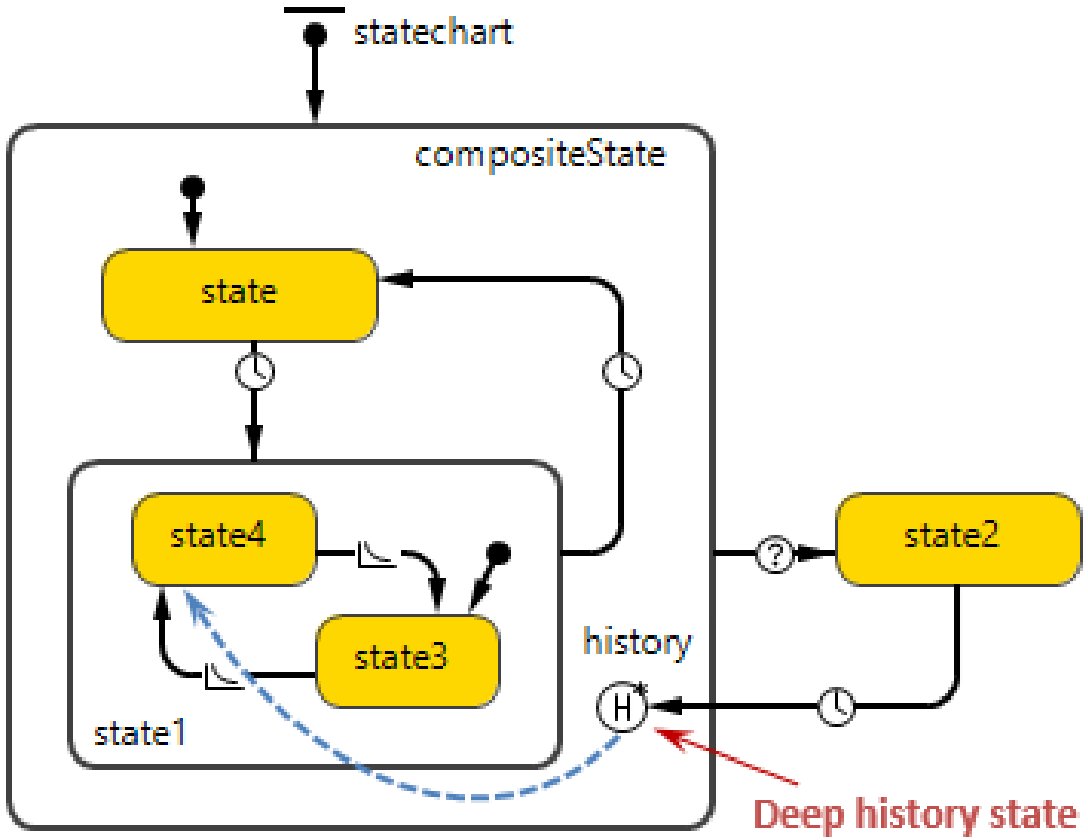
- Special vertices used to connect transitions in complex ways. They’re not real states but transition waypoints.
- Pseudostate include
  - Initial pseudostate
  - Terminate pseudostate
  - Entry point
  - Exit point
  - Choice
  - Join
  - Fork
  - Junction
  - Shallow history pseudostate
  - Deep history pseudostate

Pseudostates: History

Shallow history pseudostate represents the most recent active substate of its containing state. Notation: Circle with 'H' inside

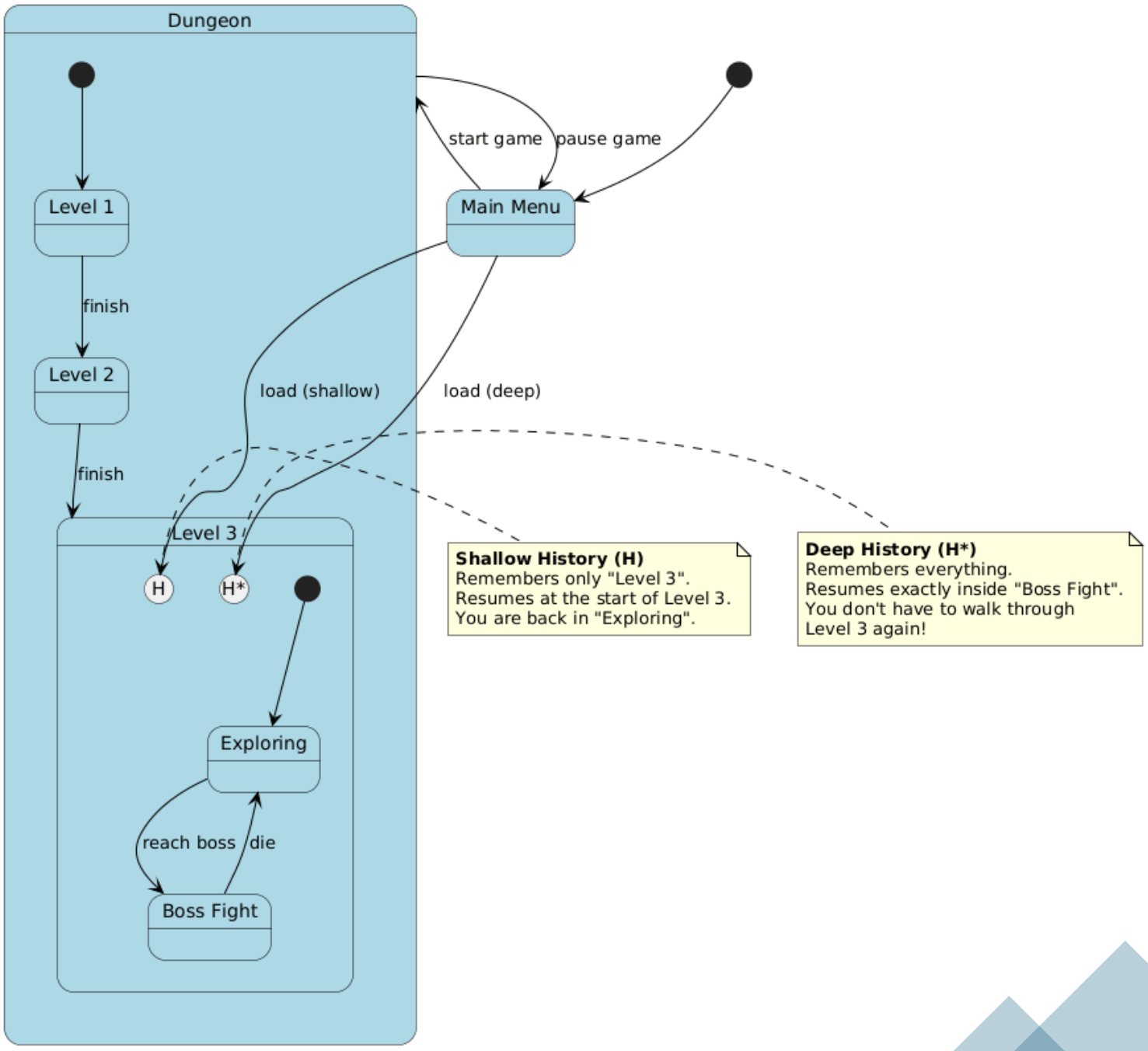
Deep history pseudostate represents the most recent active configuration of the composite state that directly contains this pseudostate. Notation: Circle with 'H*' inside





Shallow vs Deep History Pseudostate

Shallow vs Deep History Pseudostate



Complete Transition Examples

Withdraw [balance >= amount] / balance -= amount

- Trigger: withdraw event | Guard: sufficient balance | Action: deduct amount

timeout / displayError()

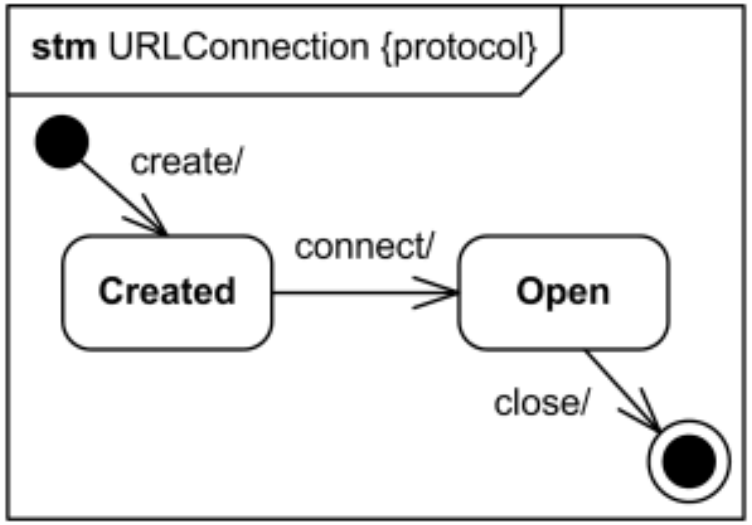
- Trigger: timeout event | No guard | Action: display error

[temperature > 100]

- No trigger (completion transition) | Guard: check temperature | No action

Protocol State Machine

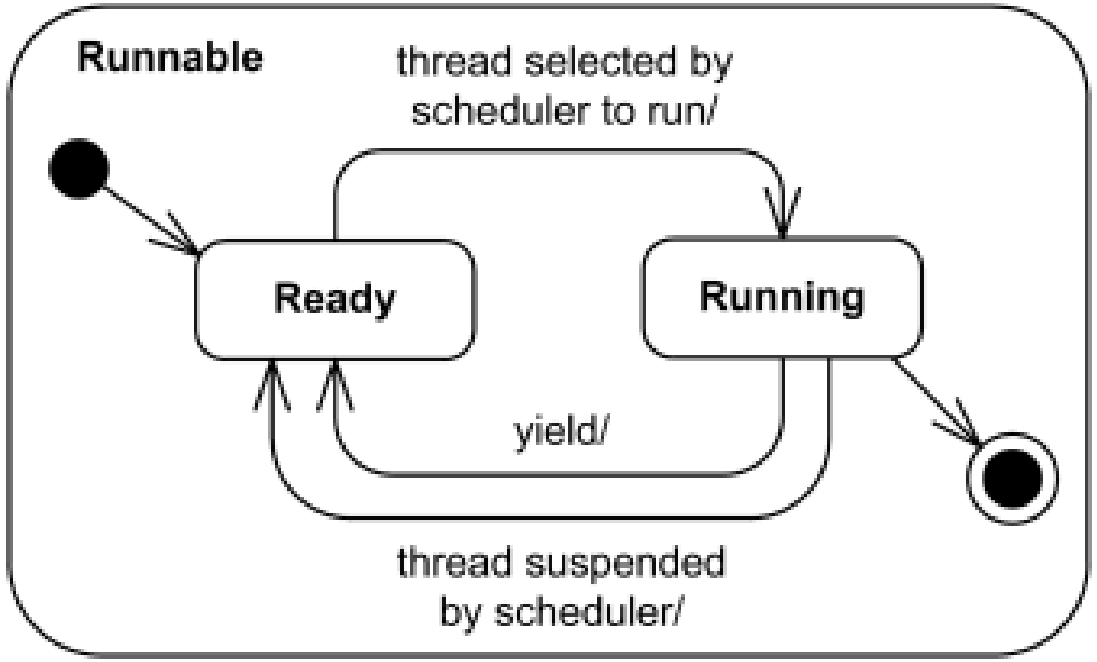
- Protocol state machine is a specialization of behavioral state machine and is used to express usage protocol or lifecycle of a class.
- It specifies which operations of the classifier can be called in which state and under which condition.
- It majorly consists of Protocol state and protocol state transitions.
- The keyword {protocol} is used to distinguish protocol state diagrams.



Protocol state machine for URLConnection class

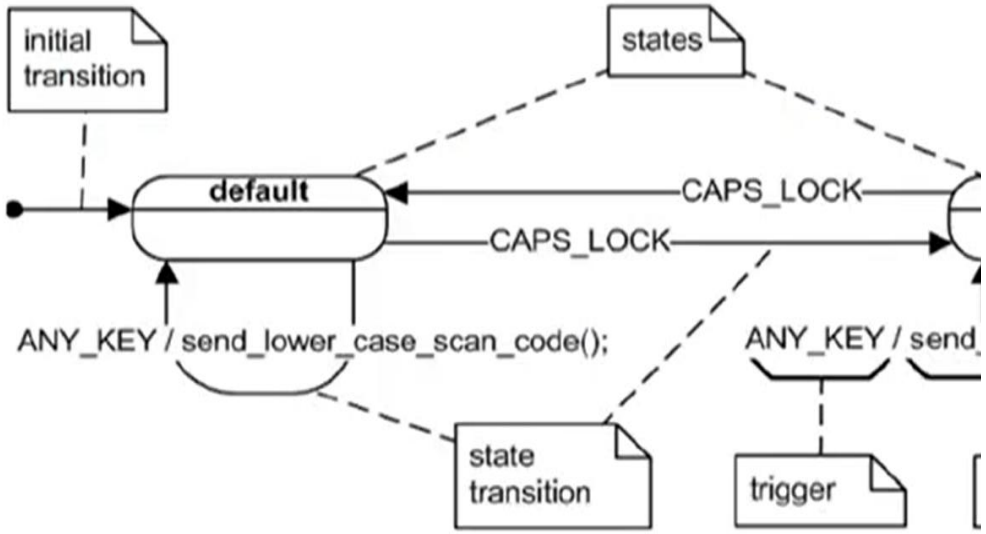
Protocol State

- Protocol state machines can have submachine states, composite states, and concurrent regions.
- Concurrent regions make it possible to express protocol where an instance can have several active states simultaneously.



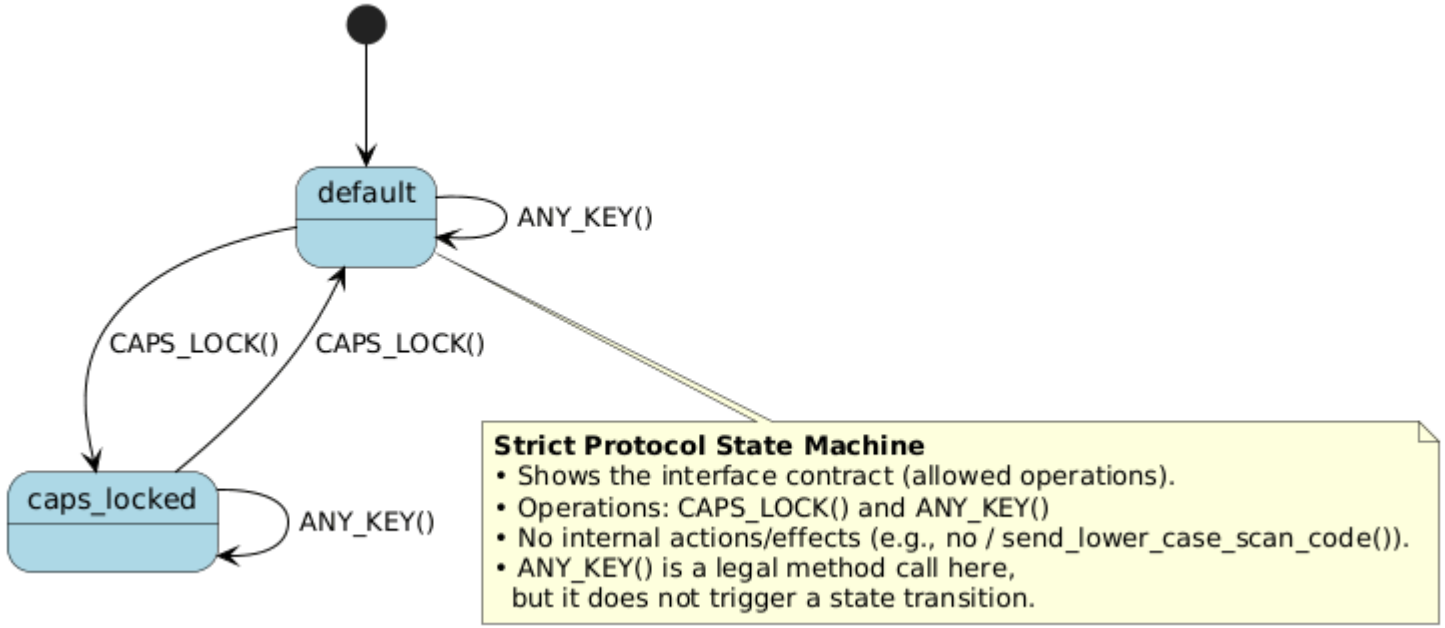
Simple composite protocol state ***Runnable****.*

Keyboard operation

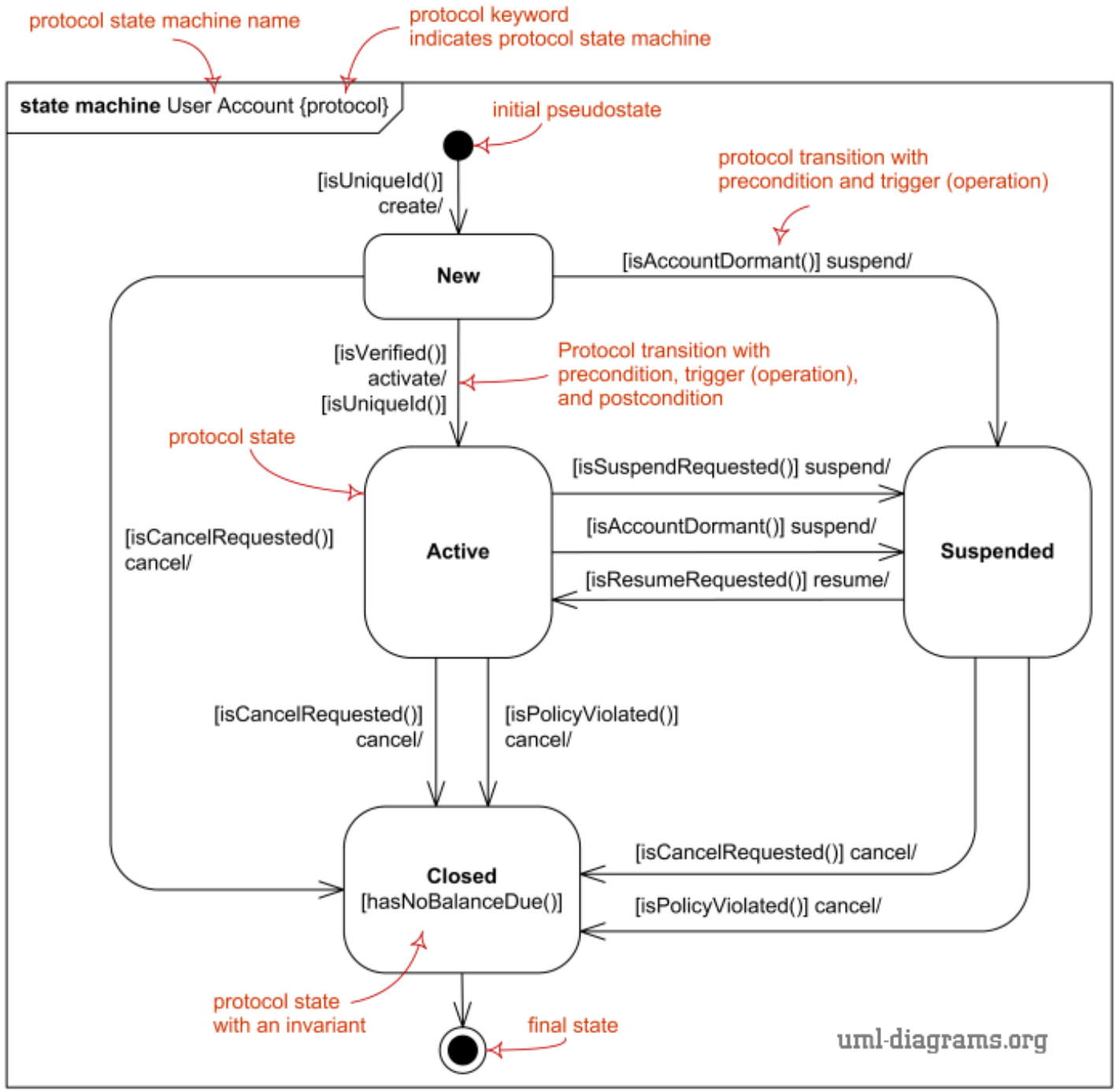


Source: *url:*

https://en.wikipedia.org/wiki/UML_state_machine#/media/File:UML_state_mach



Online Shopping User Account



Common Pitfalls and Best Practices

❌ Common Mistakes

- **Too Many States:** Creating a state for every possible value combination (state explosion)
- **Forgetting Guards:** Not specifying conditions, leading to invalid transitions
- **Confusing Protocol vs Behavioral:** Adding entry/exit actions to protocol states
- **Missing Initial State:** Composite states without initial pseudostate
- **Improper History Use:** Using history when simple default state would work
- **Overcomplicated Diagrams:** Not using submachine states for reusable patterns

✓ Best Practices

- **Name States Clearly:** Use gerunds (Waiting, Processing) or adjectives (Active, Locked)
- **Use Composite States:** Group related states to manage complexity
- **Specify All Guards:** Make transition conditions explicit
- **Document Invariants:** Note what's true while in each state
- **Consider Concurrency:** Use orthogonal regions for truly independent concerns
- **Reuse with Submachines:** Extract common patterns
- **Balance Detail:** Show enough detail to understand behavior, not every implementation detail

Real-world case study: streaming App

Main States

- **Idle:** No video loaded
- **Loading:** Buffering initial content
- **Playing:** Video actively playing
- **Paused:** Playback paused by user
- **Buffering:** Waiting for more data during playback
- **Error:** Playback error occurred

Composite State: "Playing"

The Playing state is actually composite with concurrent regions:

- **Region 1 - Video Quality:** {SD, HD, 4K} - adapts based on bandwidth
- **Region 2 - Audio:** {Normal, Muted} - independent of video quality
- **Region 3 - Captions:** {Off, On} - independent of video and audio

Key Transitions

- Idle → Loading: `loadVideo(url) / initializeBuffers()`
- Loading → Playing: `[bufferFilled >= 30%] / startPlayback()`
- Playing → Buffering: `bufferEmpty / pausePlayback()`
- Buffering → Playing: `[bufferFilled >= 10%] / resumePlayback()`
- Playing → Paused: `pauseButton / savePosition()`

Design Decisions

- **History:** Paused state uses deep history to remember quality/audio/caption settings
- **Guards:** Buffer threshold guards prevent stuttering playback
- **Entry Actions:** Playing state entry starts progress timer
- **Exit Actions:** Playing state exit saves watch position

Resources and Questions

- **Recommended Resources**
- **UML 2.5 Specification:** Official state machine semantics
- **"UML Distilled" by Martin Fowler:** Practical guide to UML diagrams
- **Online Tools:** draw.io, Lucidchart, PlantUML for creating diagrams
- **Practice:** Model everyday objects (vending machine, elevator, microwave)

- A good video for understanding: https://www.youtube.com/watch?v=qICdYjJA-Ps